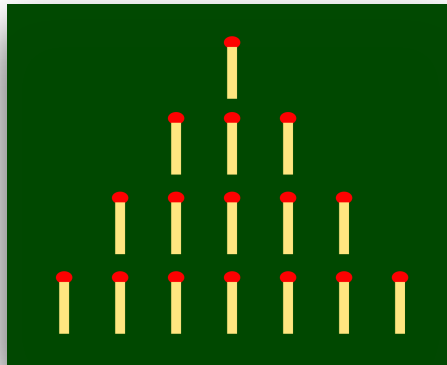


Jeu de Nim avec Intelligence Artificielle et Gestion des Statistiques



Objectif

Développer une application complète du jeu de Nim en Python. L'application devra intégrer une interface utilisateur, la gestion des joueurs, l'enregistrement des parties dans une base de données, la production de statistiques ainsi qu'une intelligence artificielle capable de jouer à différents niveaux de difficulté.

Description du jeu

Le jeu de Nim est un jeu de stratégie à deux joueurs.

Au début de la partie, plusieurs tas (ou piles) d'objets sont disposés sur le plateau. Une configuration classique comprend trois piles contenant respectivement 3, 5 et 7 objets, mais l'application devra permettre de générer ou de définir d'autres configurations.

À tour de rôle, les joueurs :

- Choisissent une seule pile ;
- Retirent un ou plusieurs objets de cette pile ;
- Ne peuvent pas retirer d'objets de plusieurs piles au cours du même tour.

Chaque coup doit retirer au minimum un objet.

Condition de victoire

La partie se termine lorsque toutes les piles sont vides.

Selon la règle classique du jeu de Nim :

- Le joueur qui retire le dernier objet remporte la partie.

Fonctionnalités attendues

1. Gestion des joueurs

L'application doit permettre :

- La création d'un profil joueur ;
- La sélection d'un joueur existant ;
- La consultation de l'historique des parties ;
- L'affichage des statistiques individuelles.

2. Modes de jeu

L'application doit proposer les modes suivants :

Mode Joueur contre Joueur (JcJ)

Deux joueurs humains s'affrontent en jouant alternativement.

Mode Joueur contre Intelligence Artificielle (JcIA)

Un joueur humain affronte l'ordinateur.

Avant le début de la partie, le joueur choisit le niveau de difficulté de l'intelligence artificielle.

3. Gestion des parties

L'application doit :

- Afficher graphiquement les piles d'objets ;
- Vérifier la validité des coups ;
- Détecter automatiquement la fin de partie ;
- Enregistrer toutes les informations relatives aux parties ;
- Permettre le démarrage d'une nouvelle partie avec différentes configurations de piles.

4. Utilisation d'une base de données

Une base de données devra stocker :

- Les informations des joueurs ;
- La date et l'heure des parties ;
- La durée des parties ;
- Le mode de jeu utilisé ;
- Le niveau de difficulté sélectionné ;
- Les configurations initiales des piles ;
- Le gagnant et le perdant ;
- Les scores cumulés ;
- Les victoires, défaites et matchs nuls éventuels.

5. Tableau de bord statistique

L'application devra permettre l'affichage :

- Du nombre total de parties jouées ;
- Du nombre de victoires et de défaites par joueur ;
- Du taux de victoire ;
- Du classement des joueurs ;
- Du temps moyen des parties ;
- Du nombre de parties jouées à chaque niveau de difficulté ;
- De l'évolution du score des joueurs.

Les statistiques pourront être présentées sous forme de tableaux ou de graphiques.

Intelligence Artificielle

Niveau 1 : Débutant

L'ordinateur joue des coups aléatoires parmi les coups autorisés.

Niveau 2 : Intermédiaire

L'ordinateur applique des règles simples afin d'améliorer ses chances de gagner.

Exemples :

- Privilégier les grandes piles ;
- Réduire progressivement le nombre total d'objets.

Niveau 3 : Avancé

L'intelligence artificielle utilise l'algorithme **Minimax** afin d'explorer les états possibles du jeu et de choisir le meilleur coup.

Niveau 4 : Expert (Bonus)

Implémentation de la stratégie mathématique optimale du jeu de Nim basée sur le calcul du **Nim-Sum** (opération XOR).

L'ordinateur devient alors pratiquement imbattable.

Contraintes techniques

Le projet devra mettre en œuvre :

- Les variables, conditions et boucles ;
- Les fonctions et modules Python ;
- **Pas de programmation orientée objet ;**
- Les structures de données (listes) ;
- La gestion des exceptions ;
- Les bases de données MySQL ;
- Une interface graphique (Tkinter ou Pygame) ;
- Les algorithmes de recherche et de prise de décision ;
- Les notions fondamentales de théorie des jeux.

Architecture proposée

```
game/
|
|— main.py
|— player.py
|— enemy.py
|— database.py
|— settings.py
|— assets/
|   |— images/
|   |— sounds/
```